

Лабораторная работа № 12

Программирование в командном процессоре ОС UNIX. Командные файлы

Студент: Чжао Синья

Учётная запись: zhaoxinya

Имя хоста: zhaoxinya

Номер студенческого билета: 113225832

Дисциплина: Архитектура компьютеров и операционные системы. Операционные системы

Содержание

1. Цель работы
 2. Задание работы
 3. Теоретические сведения
 4. Соглашения об именовании
 5. Ход выполнения работы
 6. Выводы
 7. Ответы на контрольные вопросы
 8. Библиография
-

Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать и отлаживать командные файлы (shell-скрипты) на языке bash.

Задание работы

1. Написать командный файл, выполняющий резервное копирование самого себя в каталог `~/backup` с использованием архиватора (`tar` , `zip` или `bzip2`).
2. Написать командный файл, обрабатывающий произвольное число аргументов командной строки, в том числе превышающее десять.

3. Написать командный файл — аналог команды `ls` (без использования `ls` и `dir`), выводящий информацию о каталоге и правах доступа к файлам.
4. Написать командный файл, который принимает в качестве аргументов путь к каталогу и расширение файла, и подсчитывает количество файлов с указанным расширением.

Теоретические сведения

Командные оболочки

В UNIX/Linux наиболее распространены:

- `sh` — Bourne shell, базовая оболочка;
- `csh` — C shell, C-подобный синтаксис, история команд;
- `ksh` — Korn shell, совместим с `sh`, расширен `csh`;
- `bash` — Bourne Again Shell, стандартная оболочка большинства Linux-систем.

POSIX

POSIX (Portable Operating System Interface) — набор стандартов IEEE, обеспечивающих совместимость UNIX/Linux-подобных систем и переносимость прикладных программ на уровне исходного кода.

Переменные в `bash`

```
name=value  
echo $name
```

Массивы:

```
arr=(a b c)  
echo ${arr[0]}  
echo ${arr[@]}
```

Арифметические операторы

Оператор	Описание
<code>+ - * / % **</code>	арифметика
<code><< >></code>	сдвиги
<code>& ^ ~</code>	битовые
<code>&& !</code>	логические

Арифметическая подстановка

```
i=$((i + 1))
```

Стандартные переменные

`HOME` , `PATH` , `PS1` , `PS2` , `IFS` , `TERM` , `LOGNAME` , `MAIL` , `PWD` , `USER` , `SHELL` .

Метасимволы

- `*` — произвольная строка, в том числе пустая;
- `?` — один произвольный символ;
- `[c1-c2]` — диапазон символов;
- `\` — экранирование;
- `'...'` — буквальное значение;
- `"..."` — с подстановкой переменных.

Командные файлы

Создание и запуск:

```
touch script.sh
chmod +x script.sh
./script.sh
```

Позиционные параметры

`$0` , `$1` , `$2` , ..., `$#` , `$*` , `$@` , `$?` , `$$` , `!` .

Управляющие конструкции

- `if ... then ... elif ... else ... fi`
- `case ... esac`

- `for ... do ... done`
- `while ... do ... done`
- `until ... do ... done`
- `break`, `continue`

Команда `getopts`

```
while getopts "i:o:p:Cn" opt; do
  case $opt in
    i) infile=$OPTARG;;
    o) outfile=$OPTARG;;
    p) pattern=$OPTARG;;
    C) case_sensitive=1;;
    n) show_line_numbers=1;;
  esac
done
```

Соглашения об именовании

- Пользователь ВМ: `zhaoxinya`
- Имя хоста: `zhaoxinya`
- Имя виртуальной машины: `ZhaoXinya2`

Проверка:

```
whoami
hostname
id
```

```
zhaoxinya@zhaoxinya:~$ whoami
zhaoxinya
zhaoxinya@zhaoxinya:~$ hostname
zhaoxinya
zhaoxinya@zhaoxinya:~$ id
uid=1000(zhaoxinya) gid=1000(zhaoxinya) groups=1000(zhaoxinya),10(wheel) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Ход выполнения работы

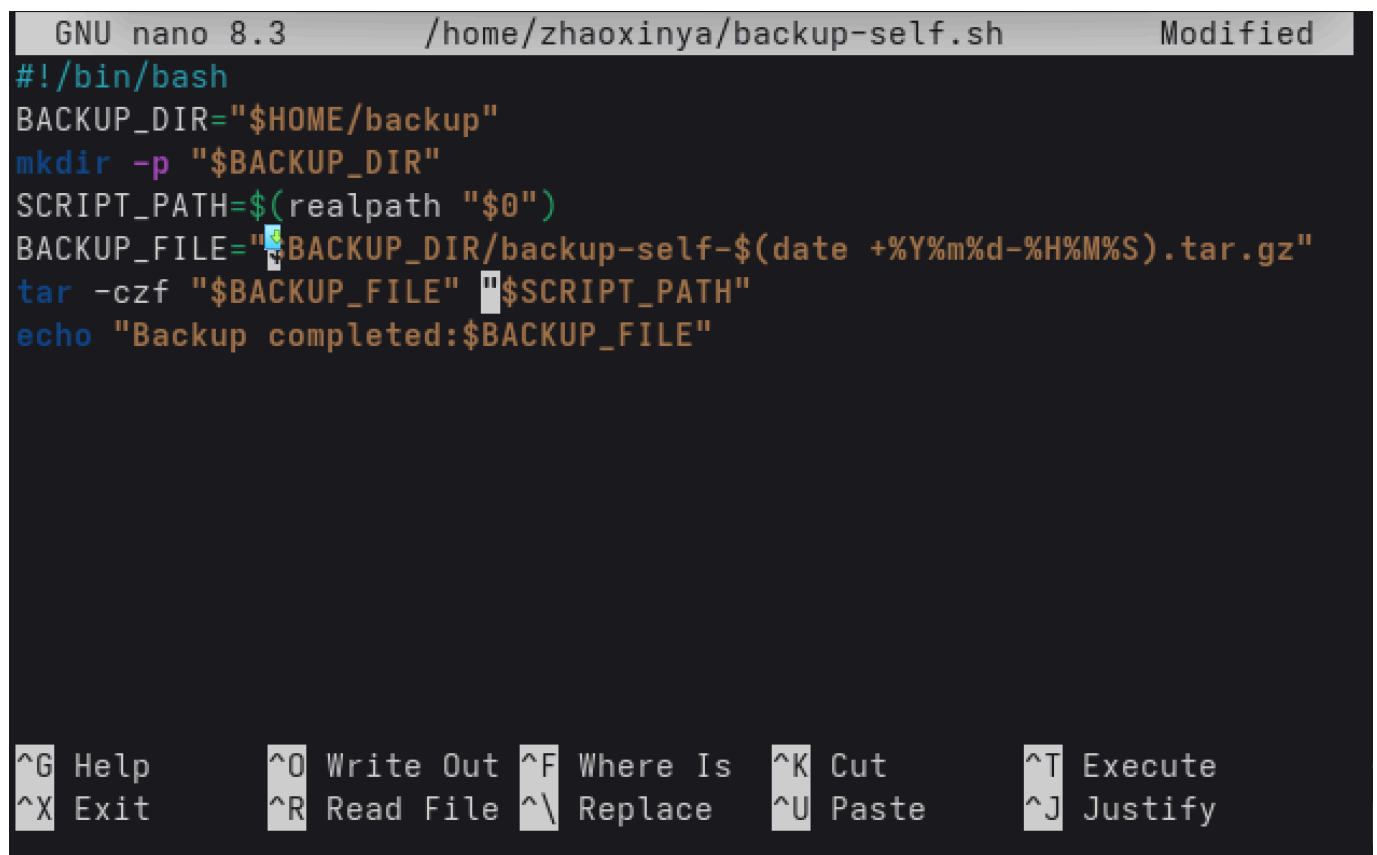
Задание 1. Резервное копирование скрипта

Создан файл `~/backup-self.sh` с помощью редактора `nano` :

```
nano ~/backup-self.sh
```

Исходный код:

```
#!/bin/bash
BACKUP_DIR="$HOME/backup"
mkdir -p "$BACKUP_DIR"
SCRIPT_PATH=$(realpath "$0")
BACKUP_FILE="$BACKUP_DIR/backup-self-$(date +%Y%m%d-%H%M%S).tar.gz"
tar -czf "$BACKUP_FILE" "$SCRIPT_PATH"
echo "Backup completed:$BACKUP_FILE"
```

A screenshot of the GNU nano 8.3 text editor. The title bar shows the editor version, the file path `/home/zhaoxinya/backup-self.sh`, and the status `Modified`. The editor content displays the same script code as the previous block. At the bottom, there is a help menu with keyboard shortcuts: `^G` Help, `^X` Exit, `^O` Write Out, `^R` Read File, `^F` Where Is, `^N` Replace, `^K` Cut, `^U` Paste, `^T` Execute, and `^J` Justify. The cursor is positioned at the end of the `echo` command line.

```
GNU nano 8.3 /home/zhaoxinya/backup-self.sh Modified
#!/bin/bash
BACKUP_DIR="$HOME/backup"
mkdir -p "$BACKUP_DIR"
SCRIPT_PATH=$(realpath "$0")
BACKUP_FILE="$BACKUP_DIR/backup-self-$(date +%Y%m%d-%H%M%S).tar.gz"
tar -czf "$BACKUP_FILE" "$SCRIPT_PATH"
echo "Backup completed:$BACKUP_FILE"

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify
```

Установлены права на выполнение:

```
chmod +x ~/backup-self.sh
```

Скрипт выполнен:

```
~/backup-self.sh
```

Первоначально при запуске возникала ошибка:

```
tar: Cowardly refusing to create an empty archive
```

Причина ошибки — в команде `tar` между двумя аргументами `"$BACKUP_FILE"` и `"$SCRIPT_PATH"` отсутствовал пробел, из-за чего они склеивались в одно имя. После добавления пробела и использования `realpath "$0"` для получения абсолютного пути скрипт заработал корректно.

Результат выполнения:

```
tar: Removing leading `/' from member names
Backup completed:/home/zhaoxinya/backup/backup-self-20261008-142508.tar.gz
```

Проверка содержимого каталога резервных копий:

```
ls -l ~/backup/
```

Результат:

```
total 4
-rw-r--r--. 1 zhaoxinya zhaoxinya 302 Oct  8 14:25 backup-self-20261008-142508.tar.gz
```

Сообщение `tar: Removing leading '/' from member names` не является ошибкой — это стандартное поведение `tar`, который удаляет ведущий `/` из путей внутри архива, чтобы распаковка не перезаписывала абсолютные пути.

```
zhaoxinya@zhaoxinya:~$ nano ~/backup-self.sh
zhaoxinya@zhaoxinya:~$ chmod +x ~/backup-self.sh
zhaoxinya@zhaoxinya:~$ ~/backup-self.sh
tar: Removing leading `/' from member names
Backup completed:/home/zhaoxinya/backup/backup-self-20261008-142508.tar.gz
zhaoxinya@zhaoxinya:~$ ls -l ~/backup/
total 4
-rw-r--r--. 1 zhaoxinya zhaoxinya 302 Oct  8 14:25 backup-self-20261008-142508.tar.gz
```

Задание 2. Обработка произвольного числа аргументов

Создан файл `~/args.sh`:

```
nano ~/args.sh
```

Исходный код:

```
#!/bin/bash
echo "Total arguments: $#"
```

```
i=1
for arg in "$@"; do
    echo "Argument $i: $arg"
    i=$((i + 1))
done
```

```
#!/bin/bash
echo "Total argument: $#"
```

```
i=1
for arg in "$@"; do
    echo "Argument $i: $arg"
    i=$((i + 1))
done
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

Установлены права и выполнен запуск с 12 аргументами:

```
chmod +x ~/args.sh
~/args.sh a b c d e f g h i j k l
```

Результат:

```
Total arguments: 12
Argument 1: a
Argument 2: b
Argument 3: c
Argument 4: d
Argument 5: e
Argument 6: f
Argument 7: g
Argument 8: h
Argument 9: i
Argument 10: j
Argument 11: k
Argument 12: l
```

Использование "\$@" позволяет корректно обрабатывать любое количество аргументов, включая более десяти, и сохранять аргументы с пробелами как единое целое.

```
zhaoxinya@zhaoxinya:~$ nano args.sh
zhaoxinya@zhaoxinya:~$ chmod +x ~/args.sh
zhaoxinya@zhaoxinya:~$ ~/args.sh a b c d e f g h i j k l
Total argument: 12
Argument 1: a
Argument 2: b
Argument 3: c
Argument 4: d
Argument 5: e
Argument 6: f
Argument 7: g
Argument 8: h
Argument 9: i
Argument 10: j
Argument 11: k
Argument 12: l
```

Задание 3. Аналог команды ls

Создан файл ~/myls.sh :

```
nano ~/myls.sh
```

Исходный код:

```
#!/bin/bash
DIR="${1:-.}"
if [ ! -d "$DIR" ]; then
    echo "Error: $DIR is not a directory."
    exit 1
fi
echo "Contents of $DIR:"
for f in "$DIR"/* "$DIR"/.[!.]*; do
    [ -e "$f" ] || continue
    if [ -d "$f" ]; then type="d"
    elif [ -f "$f" ]; then type="-"
    else type="?"
    fi
    perms=$(stat -c %A "$f")
    echo "$type $perms $(basename "$f")"
done
```

GNU nano 8.3 /home/zhaoxinya/myls.sh Modified

```
#!/bin/bash
DIR="${1:-.}"
if [ ! -d "$DIR" ]; then
echo "Error: $DIR is not a directory."
exit 1
fi
echo "Contents of $DIR:"
for f in "$DIR"/* "$DIR"/.[!.]*; do
[ -e "$f" ] || continue
if [ -d "$f" ]; then type="d"
elif [ -f "$f" ]; then type="-"
else type="?"
fi
perms=$(stat -c %A "$f")
echo "$type $perms $(basename "$f")"
```

^G Help
^O Write Out
^F Where Is
^K Cut
^T Execute
^X Exit
^R Read File
^_ Replace
^U Paste
^J Justify

```
done
█
```

^G Help
^O Write Out
^F Where Is
^K Cut
^T Execute
^X Exit
^R Read File
^_ Replace
^U Paste
^J Justify

Установлены права и выполнен запуск:

```
chmod +x ~/mys.sh  
~/mys.sh ~
```

Результат (фрагмент):

Contents of /home/zhaoxinya:

```
- -rw-rw-r-- abc1
- -rwxr-xr-x args.sh
d drwxr-xr-x backup
- -rwxr-xr-x backup-self.sh
d drwxr-xr-x bin
- -rw-r--r-- conf.txt
d drwxr-xr-x Desktop
d drwxr-xr-x Documents
d drwxr-xr-x Downloads
- -rw-r--r-- err.txt
- -rw-r--r-- file.txt
- -rw-r--r-- lab07.sh
- -rw-r--r-- lab07.sh~
- -rw-r--r-- LICENSE
- -rw-r--r-- may
d drwxr-xr-x monthly
d drwx--x--x monthly_perm
d drwxr-xr-x Music
- -rwxr-xr-x myls.sh
d drwxr-xr-x Pictures
d drwxr-xr-x Public
d drwxr-xr-x reports
d drwxr-xr-x ski.places
d drwxr-xr-x STUD1
d drwxr-xr-x Templates
- -rw-r--r-- test1.txt
d drwxr-xr-x Videos
d drwxr-xr-x work
- -rw----- .bash_history
- -rw-r--r-- .bash_logout
- -rw-r--r-- .bash_profile
- -rw-r--r-- .bashrc
d drwxr-xr-x .bashrc.d
d drwx----- .cache
d drwxr-xr-x .config
d drwx----- .emacs.d
- -rw-r--r-- .gitconfig
d drwx----- .gnupg
- -rw-r--r-- .gtkrc-2.0
d drwxr-xr-x .local
d drwxr-xr-x .mozilla
d drwx----- .password-store
d drwx----- .ssh
- -rw-r--r-- .vimrc
- -rw-r--r-- .XCompose
```

Скрипт корректно обрабатывает как обычные, так и скрытые файлы (`.` и `..` исключены шаблоном `.[!..]*`), выводит тип (`d` — каталог, `-` — файл), права доступа и имя файла.

```
zhaoxinya@zhaoxinya:~$ nano ~/mys.sh
zhaoxinya@zhaoxinya:~$ chmod +x ~/mys.sh
zhaoxinya@zhaoxinya:~$ ~/mys.sh ~
Contents of /home/zhaoxinya:
- -rw-rw-r-- abc1
- -rwxr-xr-x args.sh
d drwxr-xr-x backup
- -rwxr-xr-x backup-self.sh
d drwxr-xr-x bin
- -rw-r--r-- conf.txt
d drwxr-xr-x Desktop
d drwxr-xr-x Documents
d drwxr-xr-x Downloads
- -rw-r--r-- err.txt
- -rw-r--r-- file.txt
- -rw-r--r-- lab07.sh
- -rw-r--r-- lab07.sh~
- -rw-r--r-- LICENSE
```

```
- -rw-r--r-- may
d drwxr-xr-x monthly
d drwx--x--x monthly_perm
d drwxr-xr-x Music
- -rwxr-xr-x myls.sh
d drwxr-xr-x Pictures
d drwxr-xr-x Public
d drwxr-xr-x reports
d drwxr-xr-x ski.plases
d drwxr-xr-x STUD1
d drwxr-xr-x Templates
- -rw-r--r-- test1.txt
d drwxr-xr-x Videos
d drwxr-xr-x work
- -rw----- .bash_history
- -rw-r--r-- .bash_logout
- -rw-r--r-- .bash_profile
- -rw-r--r-- .bashrc
```

```
d drwxr-xr-x .bashrc.d
d drwx----- .cache
d drwxr-xr-x .config
d drwx----- .emacs.d
- -rw-r--r-- .gitconfig
d drwx----- .gnupg
- -rw-r--r-- .gtkrc-2.0
d drwxr-xr-x .local
d drwxr-xr-x .mozilla
d drwx----- .password-store
d drwx----- .ssh
- -rw-r----- .vboxclient-clipboard-tty2-control.pid
- -rw-r----- .vboxclient-draganddrop-tty2-control.pid
- -rw-r----- .vboxclient-hostversion-tty2-control.pid
- -rw-r----- .vboxclient-seamless-tty2-control.pid
- -rw-r--r-- .vimrc
- -rw-r--r-- .XCompose
```

Задание 4. Подсчёт файлов по расширению

Создан файл `~/count-ext.sh` :

```
nano ~/count-ext.sh
```

Исходный код:

```
#!/bin/bash
if [ $# -ne 2 ]; then
    echo "Usage: $0 <directory> <extension>"
    exit 1
fi
DIR="$1"
EXT="$2"
if [ ! -d "$DIR" ]; then
    echo "Error: Not a directory: $DIR"
    exit 1
fi
COUNT=0
for f in "$DIR"/*. "$EXT"; do
    [ -e "$f" ] || continue
    COUNT=$((COUNT + 1))
done
echo "Files with extension .$EXT in $DIR: $COUNT"
```

```
GNU nano 8.3 /home/zhaoxinya/count-ext.sh Modified
#!/bin/bash
if [ $# -ne 2 ]; then
echo "Usage:$0 <directory> <extension>"
exit 1
fi
DIR="$1"
EXT="$2"
if [ ! -d "$DIR" ];then
echo "Error: Not a directory: $DIR"
exit 1
fi
COUNT=0
for f in "$DIR"/*. "$EXT"; do
[ -e "$f" ] || continue
COUNT=$((COUNT + 1))
done
echo "Files with extension .$EXT in $DIR: $COUNT"

[ Wrote 18 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
done
echo "Files with extension .$EXT in $DIR: $COUNT"

[ Wrote 18 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Установлены права и выполнены проверки:

```
chmod +x ~/count-ext.sh
~/count-ext.sh ~ sh
~/count-ext.sh ~ txt
```

Результат:

```
Files with extension .sh in /home/zhaoxinya: 5
Files with extension .txt in /home/zhaoxinya: 4
```

Первоначально в последней строке была ошибка `echo"Files..."` (без пробела между `echo` и `"`), из-за чего bash пытался выполнить команду с именем `echoFiles...`. После добавления пробела скрипт стал работать корректно.

```
zhaoxinya@zhaoxinya:~$ nano ~/count-ext.sh
zhaoxinya@zhaoxinya:~$ chmod +x ~/count-ext.sh
zhaoxinya@zhaoxinya:~$ ~/count-ext.sh ~ sh
Files with extension .sh in /home/zhaoxinya: 5
zhaoxinya@zhaoxinya:~$ ~/count-ext.sh ~ txt
Files with extension .txt in /home/zhaoxinya: 4
```

Выводы

В ходе лабораторной работы были изучены:

- переменные в bash и оператор `$` ;
- арифметические операции через `$( ( ))` ;
- стандартные переменные оболочки;
- метасимволы и их экранирование;
- создание и запуск командных файлов;
- позиционные параметры и команда `getopts` ;
- управляющие конструкции (`if` , `case` , `for` , `while` , `until`).

Написаны и отлажены 4 командных файла: `backup-self.sh` , `args.sh` , `mysls.sh` , `count-ext.sh` . В процессе работы были выявлены и устранены две типичные ошибки: отсутствие пробела между аргументами `tar` и отсутствие пробела между `echo` и строкой. Обе ошибки связаны с особенностями разбора командной строки bash.

Ответы на контрольные вопросы

1. Понятие командной оболочки

Командная оболочка (shell) — программа, обеспечивающая взаимодействие пользователя с операционной системой через команды. Примеры: `sh` , `csh` , `ksh` , `bash` .

2. Что такое POSIX?

POSIX — набор стандартов описания интерфейсов взаимодействия ОС и прикладных программ, разработан IEEE для совместимости UNIX/Linux-подобных систем.

3. Как определяются переменные и массивы в bash?

Переменные:

```
name=value
```

Массивы:

```
arr=(a b c)
echo ${arr[0]}
echo ${arr[@]}
```

4. Назначение операторов `let` и `read`

- `let` — выполнение арифметических операций над переменными;
- `read` — чтение строки со стандартного ввода и присваивание её переменной.

5. Арифметические операторы в bash

```
+ - * / % ** << >> & | ^ ~ && || ! .
```

6. Что означает `$(( ))`?

Арифметическая подстановка: содержимое вычисляется как арифметическое выражение, результат подставляется в командную строку.

7. Стандартные имена переменных

```
HOME , PATH , PS1 , PS2 , IFS , TERM , LOGNAME , MAIL , PWD , OLDPWD , SHELL , USER .
```

8. Что такое метасимволы?

Символы, имеющие специальное значение для shell: `* ? [] \ ' " $; & | < > () { } ``.

9. Как экранировать метасимволы?

- `\` перед символом — экранирует один символ;
- `'...'` — экранирует всё содержимое;
- `"..."` — экранирует всё, кроме `$`, ```, `\`, `!`.

10. Как создавать и запускать командные файлы?

```
touch script.sh
chmod +x script.sh
./script.sh
```

11. Как определяются функции в bash?

```
function name {
    commands
}
# или
name() {
    commands
}
```

12. Как выяснить, является ли файл каталогом?

```
[ -d "$file" ] && echo "directory"
```

13. Назначение команд `set`, `typeset`, `unset`

- `set` — показать/установить переменные и опции shell;
- `typeset` — объявить переменную с атрибутами;
- `unset` — удалить переменную.

14. Как передаются параметры в командный файл?

Позиционные параметры: `$1`, `$2`, ..., `$#`, `$*`, `$@`.

15. Специальные переменные bash

`$?` — код завершения последней команды, `$$` — PID текущего shell, `$_` — PID последнего фонового процесса, `$#` — число аргументов, `$*` / `$@` — все аргументы, `$-` — флаги shell, `$_` — последний аргумент предыдущей команды.

Библиография

1. Курячий Г. В., Маслинский К. А. Операционная система Linux. — М.: Интуит.Ру, 2005. — 392 с.
2. Робачевский А. Операционная система UNIX. — СПб.: БХВ-Петербург, 2010. — 656 сс.
3. Robbins A. Bash Pocket Reference. — O'Reilly Media, 2016. — 156 сс.